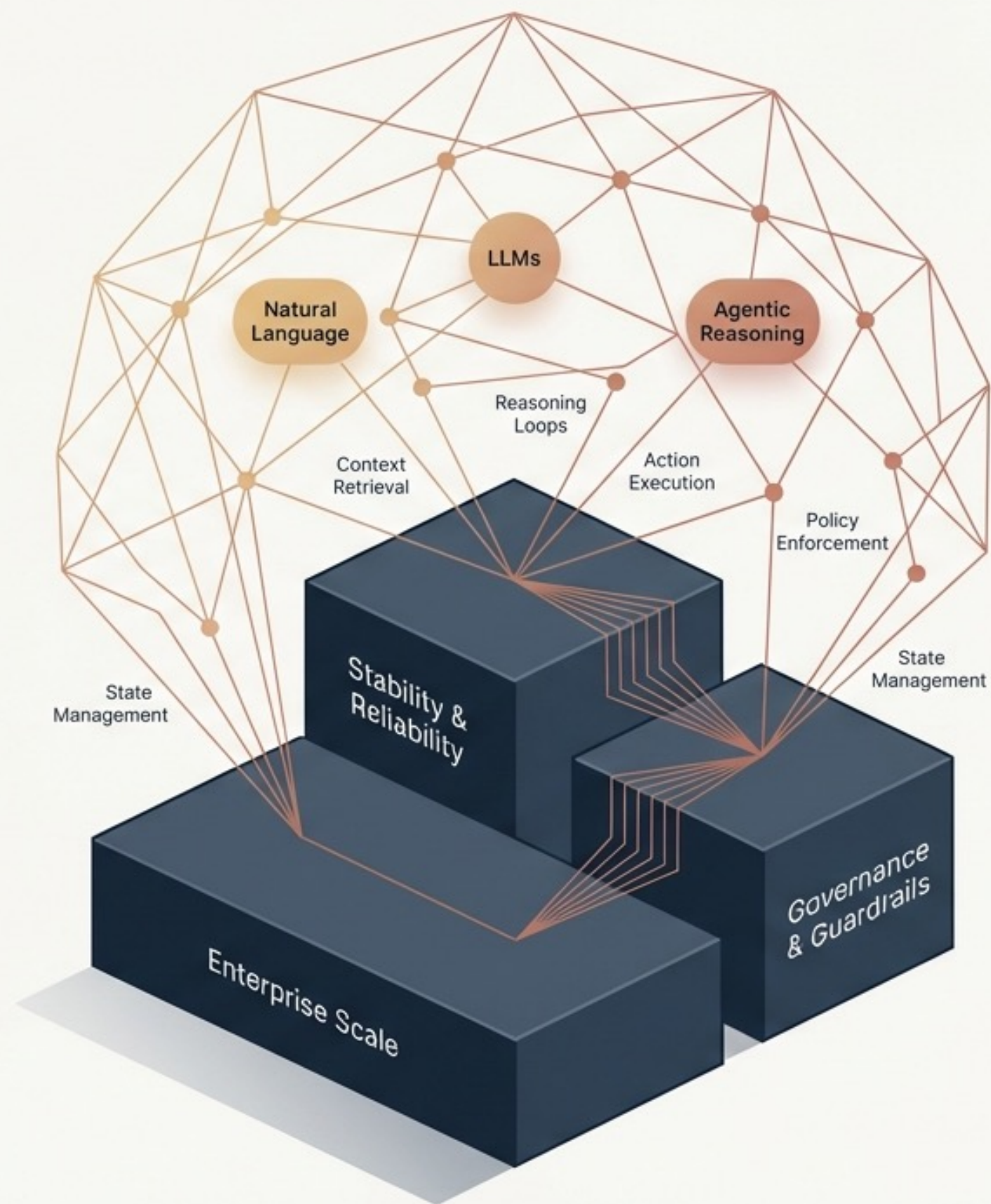


Config-Driven Agentic Platform

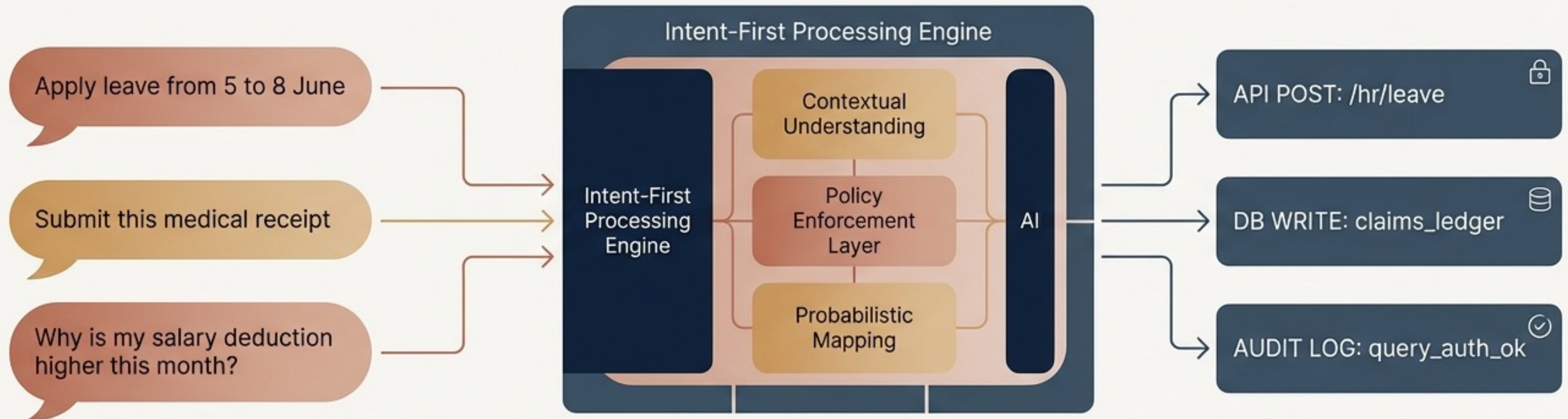
Enterprise Reference Architecture using
LangGraph4j, Spring AI, and MCP.



Translating natural language intents into governed, auditable actions

The Goal

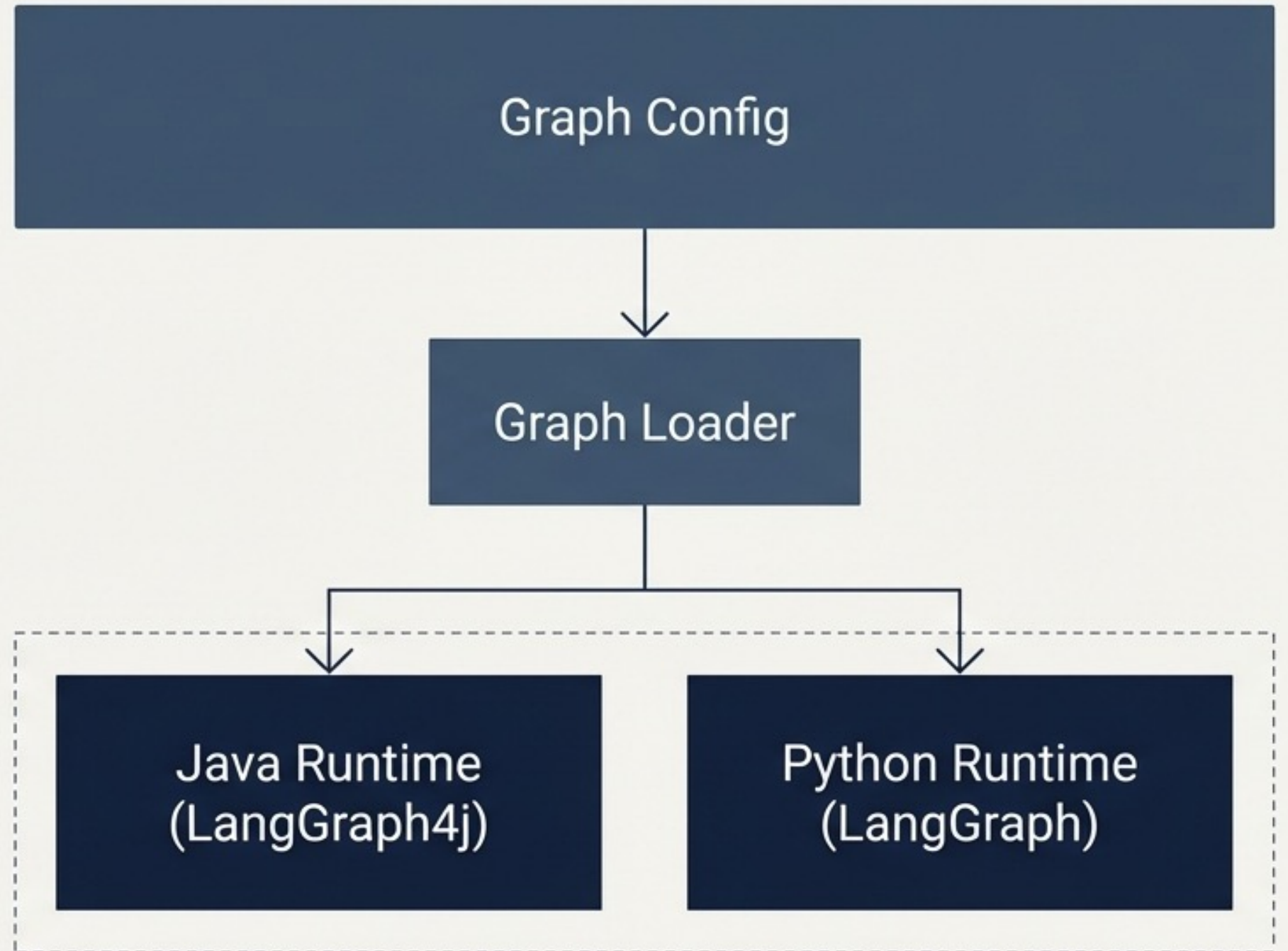
Enable AI-assisted employee workflows while maintaining strict enterprise governance, safe execution boundaries, and future portability.



Graphs are data, not code

Workflows are defined entirely in configuration, allowing absolute portability across runtimes.

- Graph Config: YAML configuration dictates the workflow.
- Runtime: Acts purely as the execution engine.
- Nodes: Function as plug-and-play extensions.



Five bank-grade principles ensure safe execution

1. AI Proposes

Agents can only suggest actions, never execute writes autonomously.

2. Governance Executes

Only deterministic services possess the authority to execute write operations.

3. Tool Boundary

All enterprise APIs are strictly invoked through the Model Context Protocol (MCP).

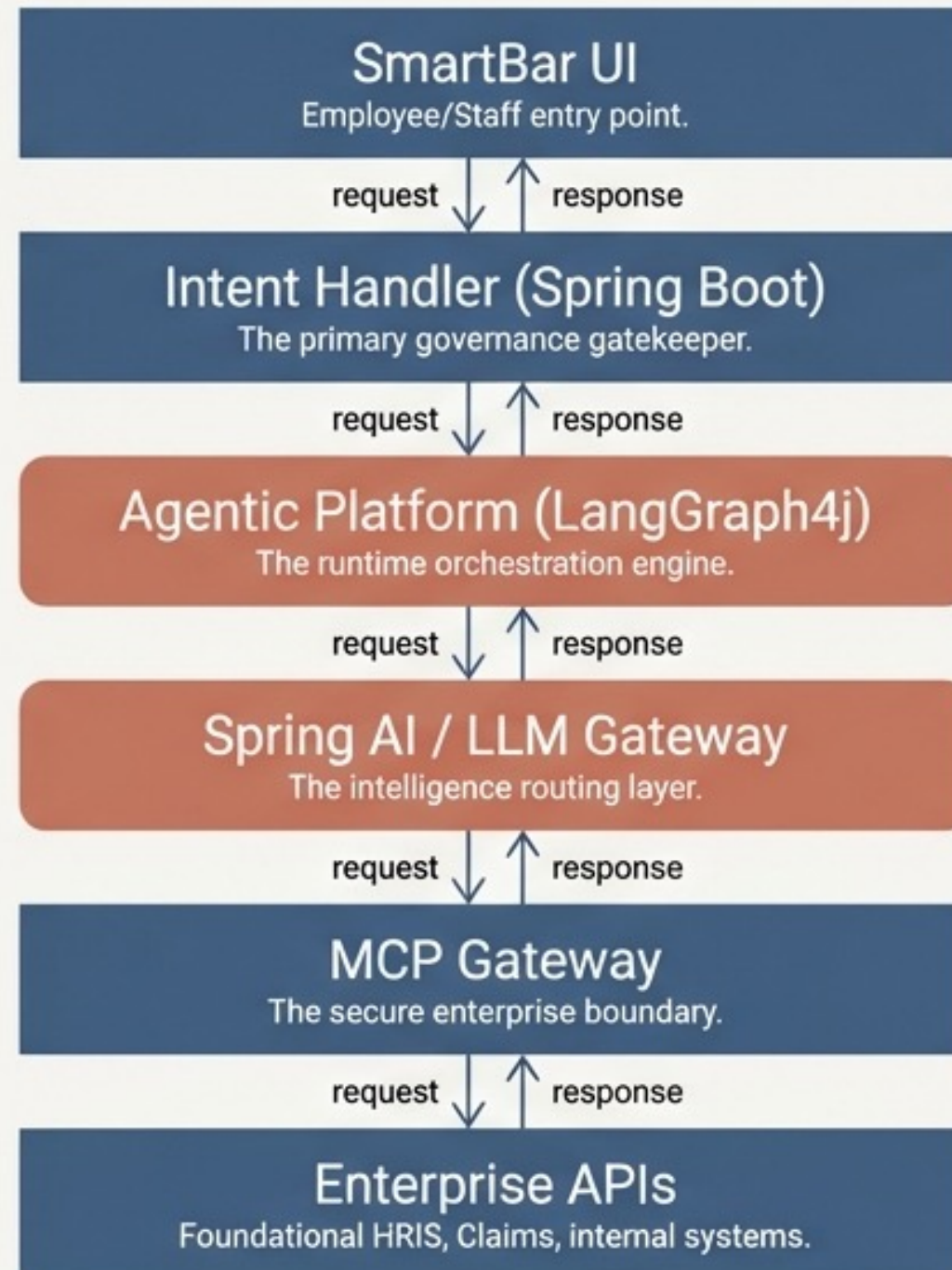
4. Event Durability

Long-running tasks are handled asynchronously to prevent timeouts and data loss.

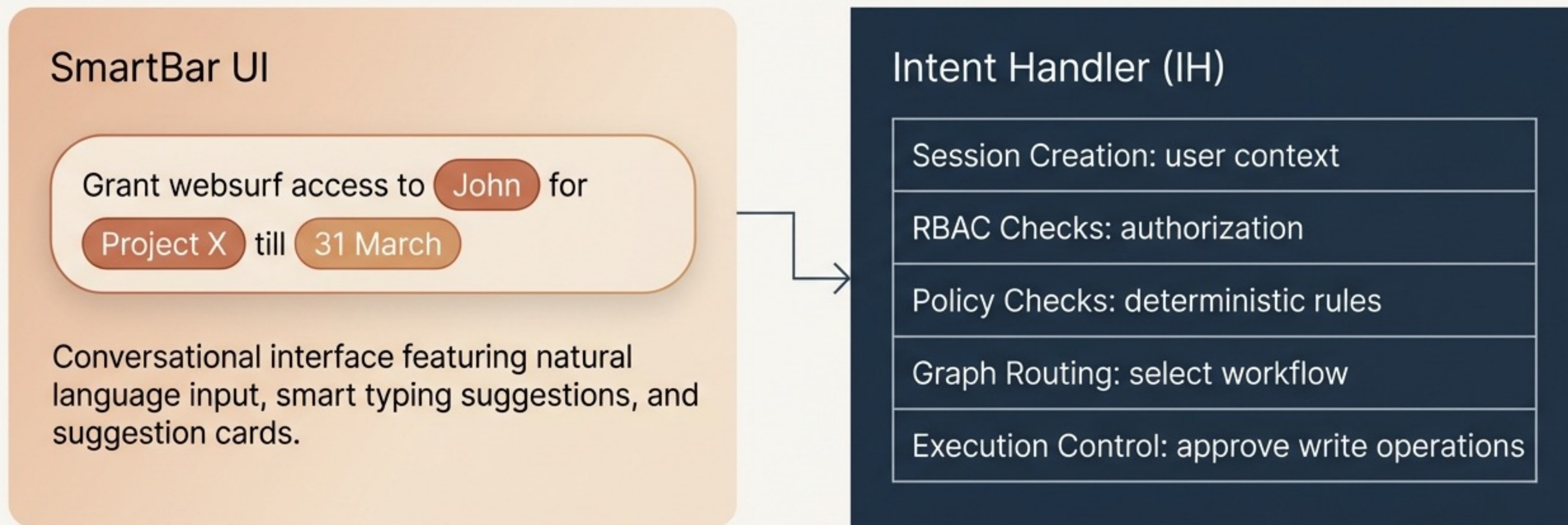
5. Auditability

Every single step, evaluation, and tool call is permanently logged.

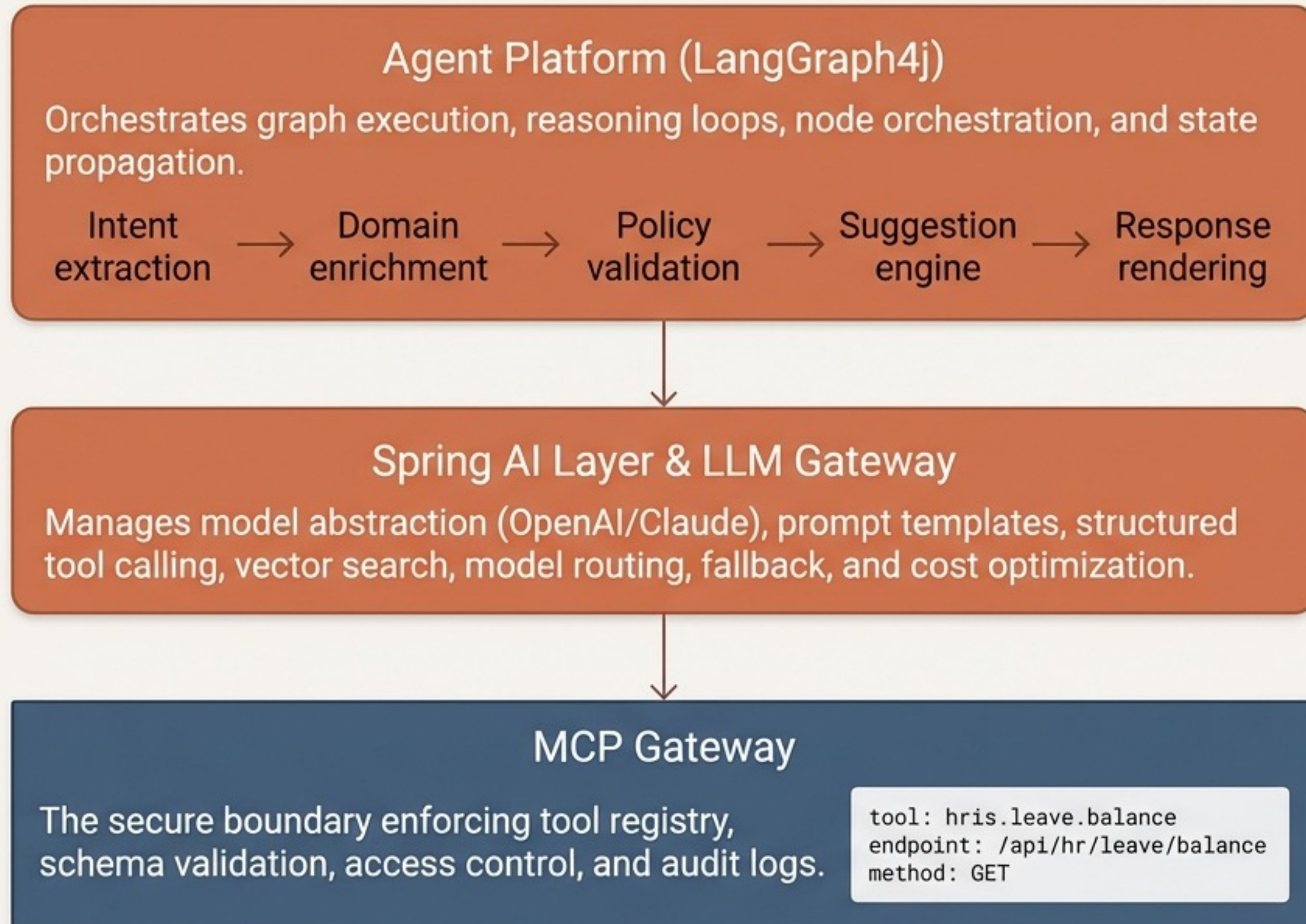
The complete vertical system stack



Orchestrating interactions and enforcing strict authorization



Secure reasoning loops and enterprise API boundaries



Programming the platform through configuration artifacts

Graph Configuration (YAML)

Defines the actual node-to-node workflow.

```
graphId: leave_apply_v1
nodes:
  id: A_INTENT
  type: intent_extractor
  next: ENRICH_HRIS
```

Tool Registry (Postgres)

Catalogs available enterprise APIs and access methods.

tool_id	endpoint	method	type
hris.leave.balance	/leave/balance	GET	READ

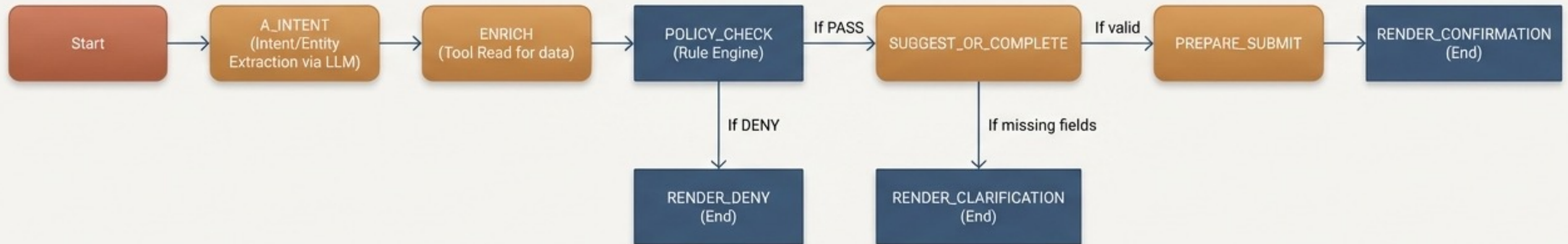
Rules Engine (JSON)

Enforces deterministic business policies.

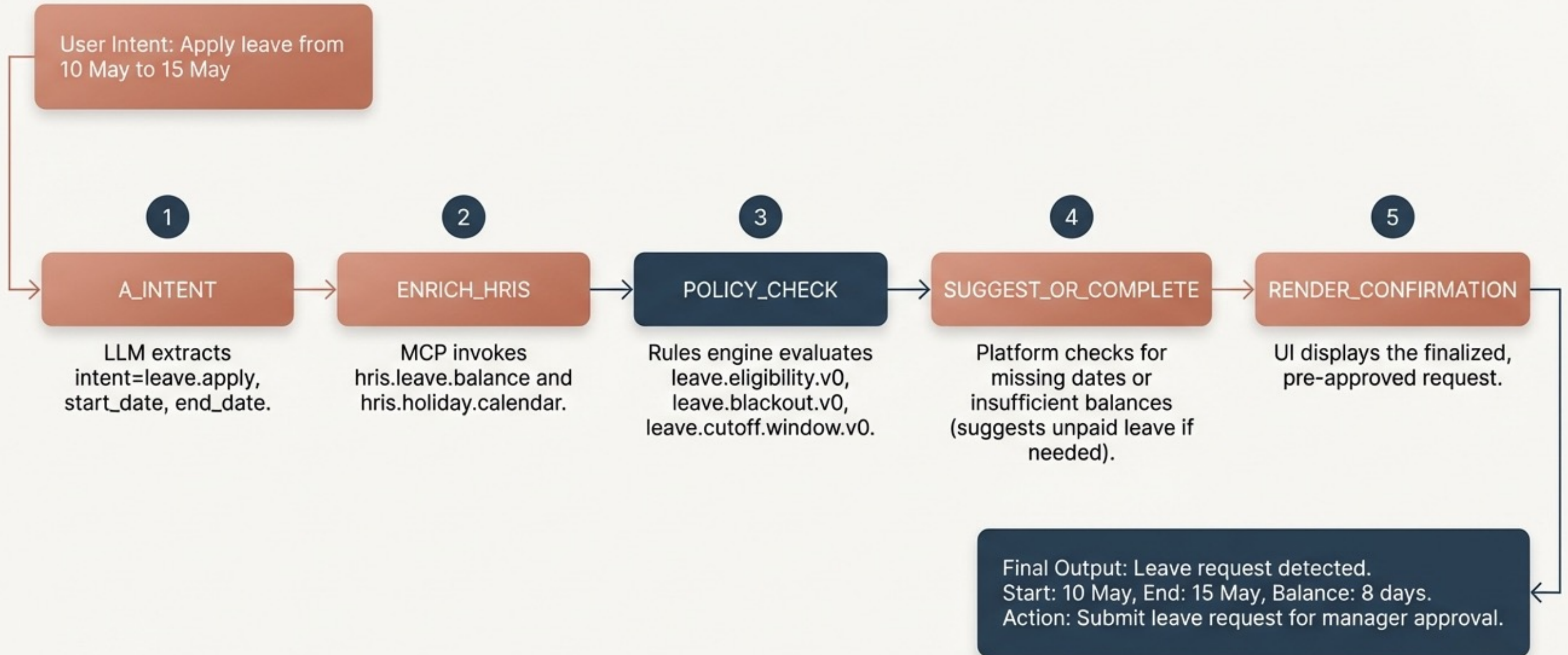
```
{
  "rule_id": "leave.eligibility.v0",
  "condition": "leave_balance >= requested_days",
  "deny_reason": "INSUFFICIENT_BALANCE"
}
```

Defining the agent collaboration paths

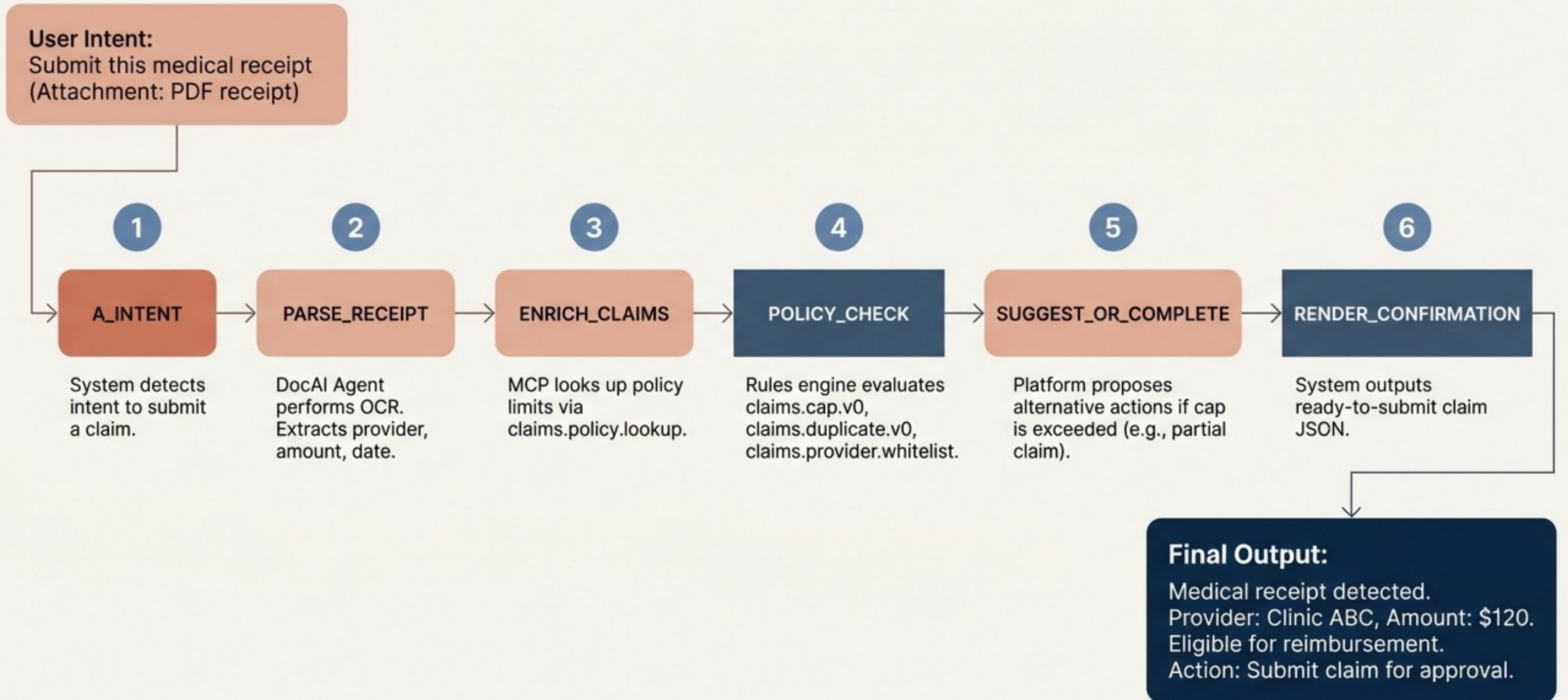
Graph nodes represent specific functional plugins; edges represent routing based on state and policy evaluations.



Workflow execution map: Leave application



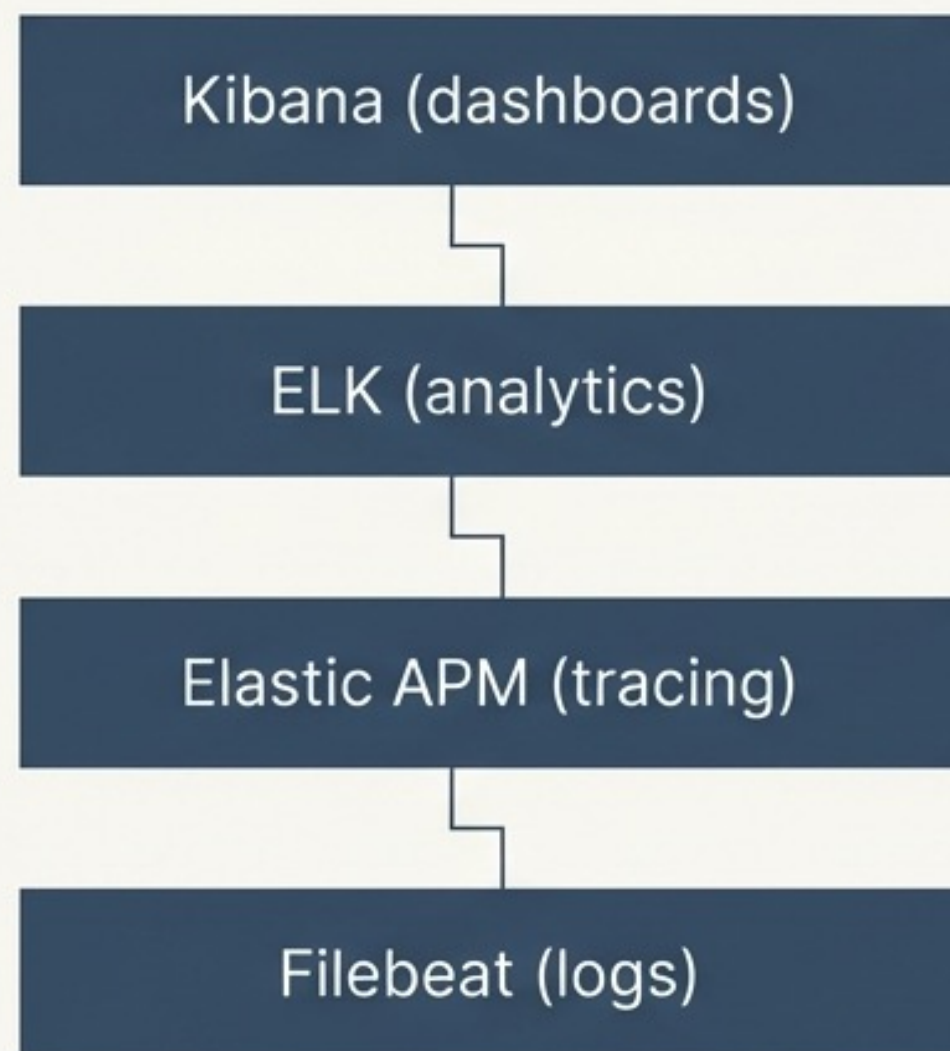
Workflow execution map: Multimodal claim submission



Complete telemetry and platform observability

High visibility into both system performance and AI reasoning loops is required for enterprise operation.

The Monitoring Stack



Key Platform Metrics

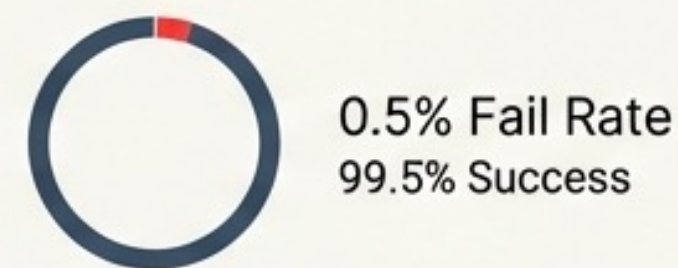
LLM Latency

Tracking response times across models



Tool Failures

Tracking MCP endpoint reliability



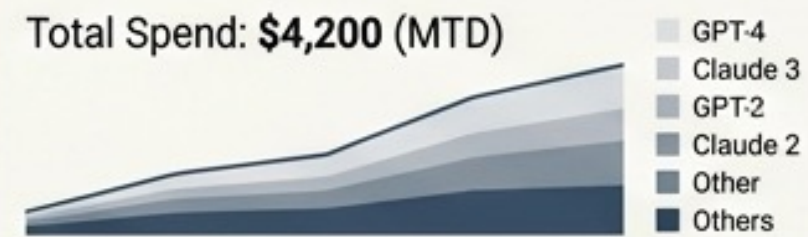
Suggestion Acceptance Rate

Tracking NBA engine quality



Model Cost

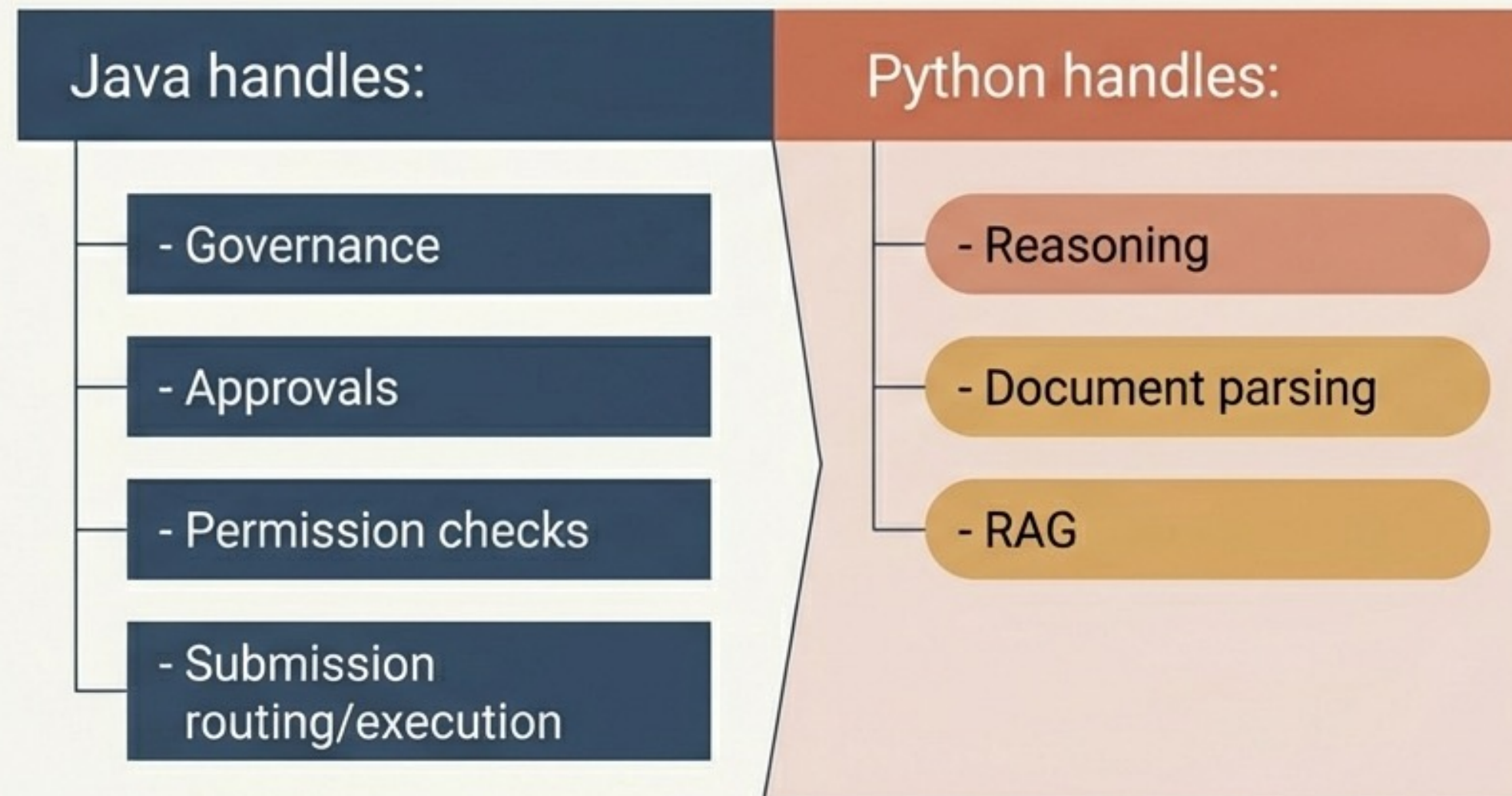
Tracking token spend across routed models



A hybrid migration strategy for immediate deployment

Because workflows are configuration-driven, moving runtimes requires low-to-medium effort.

Phase 1 - Hybrid Approach



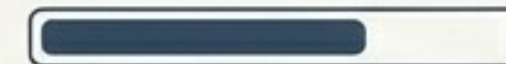
Migration Effort

Java -> Hybrid Python



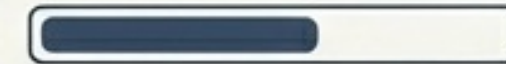
Low Effort

Hybrid -> Python Dominant



Medium Effort

On-prem -> Cloud



Medium Effort

Strategic independence and zero lock-in

The platform guarantees low coupling; individual agent nodes and capabilities can evolve entirely independently of the core system.

Problem

Language Lock-in

Vendor Lock-in

Cloud Lock-in

Mechanism

**Avoided by ->
Config Graphs**

**Avoided by ->
MCP Tool Layer**

**Avoided by ->
Portable Runtimes**

Solution Details

Logic lives in YAML,
interchangeable Java/Python
runtimes.

Standardized API schemas
abstract away the underlying
enterprise vendor.

Can deploy on OpenShift,
AWS, or on-prem
interchangeably.

Stable foundations for an evolving intelligence layer

The overarching architectural mandate: strictly separate the stable definition layer from the rapidly evolving execution layer. Everything else can, and will, change.

